

Programation Orientée Objet en Java

Séance : 03

ISCAE

Dr. Mohamed Ould Djibril

November 12, 2018

Institut Supérieur de Comptabilité et d'Administration
des Entreprises - ISCAE

Objectifs du cours d'aujourd'hui

Nous allons ...

- Introduction au langage Java
- Les classes et les objets en Java
- Prise en main d'un IDE (TP)

introduction

Rappel

- Java est un langage orienté objet: l'entité de base de tout code Java est la classe
- Créé en 1995 par Sun Microsystems
- Sa syntaxe est proche du langage C
- Il est fourni avec le JDK (Java Development Kit)
 - Outils de développement
 - Ensemble de paquetages très riches et très variés
- Multi-tâches (threads)
- Portable (machine virtuelle)

introduction

Rappel

- Java est un langage orienté objet: l'entité de base de tout code Java est la classe
- Créé en 1995 par Sun Microsystems
- Sa syntaxe est proche du langage C
- Il est fourni avec le JDK (Java Development Kit)
 - Outils de développement
 - Ensemble de paquetages très riches et très variés
- Multi-tâches (threads)
- Portable (machine virtuelle)

Introduction

Gratuit

- Le 13 novembre 2006, Sun annonce le passage de Java, c'est-à-dire le JDK (JRE et outils de développement) sous licence GPL.

Le JDK

- L'outil de base pour développer en Java
- Une adresse: <http://java.sun.com>.
- Dernière version : 1.7 ou java 7

Introduction

Evolution

- Java n'est pas un langage normalisé et il continu d'évoluer.
- Cette évolution se fait en ajoutant de nouvelle API,
- mais aussi en modifiant la machine virtuelle.

Versions successives

1.0 → 1.1 → 1.2 → 1.3 → 1.5 → 1.6 → 1.7

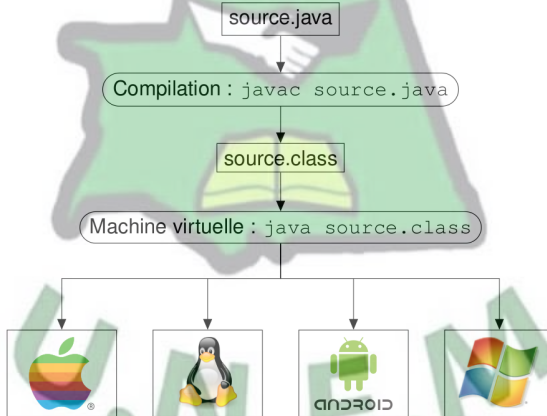
de 1.2 à 1.4 Java2

1.5 java 5

1.6 java 6 etc.

La machine virtuelle

Write once, run anywhere



Création de classes

- Pour créer une classe (un nouveau type d'objets) on utilise le mot clé **class**.
- La syntaxe pour créer une classe de nom *ClasseTest* est la suivante:

```
1 class ClasseTest{ /* nom de la class*/  
2     /* Corps de la classe  
3     attributs  
4     methodes  
5     */  
6 }
```

Ajout des commentaires

Pour commenter

- // pour un commentaire sur une seule ligne
- /* pour un commentaire étalé sur plusieurs lignes, et doit terminer avec */

Création de classes

EXEMPLE:

```
1  class Rectangle {  
2      int longueur;  
3      int largeur;  
4      int x;  
5      int y;  
6      void deplace(int dx, int dy) {  
7          x = x+ dx;  
8          y = y + dy;  
9      }  
10     int surface() {  
11         return longueur * largeur;  
12     }  
13 }
```

Quelques règles pratiques

Règles:

- La première lettre du nom d'une classe doit toujours être une lettre majuscule (ex : Chat).
- Mélange de minuscule, majuscule avec la première lettre de chaque mot en majuscule (ex : ChatGris).
- Une classe se trouve dans un fichier portant son nom suivi l'extention .java (ex : ChatGris.java)

Création et manipulation des objets

rappel

- Une fois la classe est définie, on peut créer des objets (variables)
- Chaque objet est une variable d'une classe. Il a son espace mémoire. Il admet une valeur propre à chaque attribut.
- Les valeurs des attribut caractérise l'état de l'objet.
- L'opération de création de l'objet est appelé une **instanciation**.
- Un objet est aussi appelé **une instance d'une classe**.
- On parle indifféremment d'**instance**, de **référence** ou d'**objet**



Création et manipulation des objets

Etapas de création des Objets

En deux étapes :

- 1 Déclaration de l'objet
- 2 Création de l'objet

Création et manipulation des objets

La déclaration: Syntaxe

```
1      NomDeClasse objetId;  
2      NomDeClasse objetId1, objetId2 ,....;
```

Attention

- ❶ La déclaration d'une variable de type primitif réserve un emplacement mémoire pour stocker la variable
- ❷ Par contre, la déclaration d'un objet, ne réserve pas une place mémoire pour l'objet, mais seulement un emplacement pour une référence à cet objet.
- ❸ un objet seulement déclaré vaut "null".

Création et manipulation des objets

La création effective d'un objet

- La création doit être demandé explicitement dans le programme en faisant appel à l'opérateur **new**.
- La création *réserve de la mémoire pour stocker l'objet* et initialise les attributs.

La création d'un objet : syntaxe

1

```
new NomDeClasse()
```

Création et manipulation des objets

Exemple:

Considérons la classe suivante:

```
1  class ClasseTest {  
2      /* Corps de la classe ClasseTest */  
3  }
```

ClassTest objA ; /* déclare une référence sur l'objet objA */
objA= new ClasseTest(); /* crée un emplacement pour
stocker l'objet objA */

On peut également écrire tout simplement:

ClassTest objA = new ClasseTest();

La création d'objets identiques

```
1 MaClasse m1 = new MaClasse();  
2 MaClasse m2 = m1;
```

Résultat:

- m1 et m2 contiennent la même référence et pointent donc tous les deux sur le même objet
- les modifications faites à partir d'une des variables modifient l'objet.

La création d'objets identiques

```
1 MaClasse m1 = new MaClasse();  
2 MaClasse m2 = m1.clone();
```

Résultat:

- m2 est une copie identique de m1
- m1 et m2 ne contiennent plus la même référence et pointent donc sur des objets différents.
- la méthode **clone()** est héritée de la classe **Object** qui est la classe mère de toutes les classes en Java..

Les références et la comparaison d'objets

Attention !

- Les variables de type objet que l'on déclare ne contiennent pas un objet mais une référence vers cet objet.
- L'opérateur `==` compare ces références.

```
1      Rectangle r1 = new Rectangle(100,50);
2      Rectangle r2 = new Rectangle(100,50);
3      if (r1 == r1) { ... } // vrai
4      if (r1 == r2) { ... } // faux
```

Les références et la comparaison d'objets

Comparaison correcte

- Pour comparer l'égalité des variables de deux instances, il faut munir la classe d'une méthode à cet effet : la méthode **equals()** héritée de **Object**.
- Pour s'assurer que deux objets sont de la même classe, il faut utiliser la méthode **getClass()** de la classe **Object**.

```
1 (obj1.getClass().equals(obj2.getClass()))
```

L'objet "null"

null

- L'objet null est utilisable partout.
- Il n'appartient pas à une classe mais il peut être utilisé à la place d'un objet de n'importe quelle classe ou comme paramètre.
- Ne peut pas être utilisé comme un objet normal : il n'y a pas d'appel de méthodes et aucune classe ne peut en hériter.
- Le fait d'initialiser à null une variable référençant un objet permet au **ramasse miette** de libérer la mémoire allouée à l'objet.

La variable "this"

this

- Cette variable sert à référencer dans une méthode l'instance de l'objet en cours d'utilisation.
- **this** est un objet qui est égal à l'instance de l'objet dans lequel il est utilisé.
- **this** est aussi utilisé quand l'objet doit appeler une méthode en se passant lui même en paramètre de l'appel.

```
1 private int nombre;  
2 public maclasse(int nombre) {  
3     nombre = nombre; /*variable de classe = ↔  
4     variable en parametre du constructeur */  
5 }
```

L'opérateur "instanceof"

this

L'opérateur **instanceof** permet de déterminer la classe de l'objet qui lui est passé en paramètre.

```
1 void testClasse(Object o) {  
2     if (o instanceof MaClasse )  
3         System.out.println(" o est une instance de ←  
4             la classe MaClasse ");  
5     else  
6         System.out.println(" o n'est pas un objet ←  
7             de la classe MaClasse ");  
8 }
```

L'opérateur "instanceof"

```

1 void afficheChaine(Object o) {
2     if (o instanceof MaClasse)
3         System.out.println(o.getChaine());
4         // erreur a la compilation car la
5         // methode getChaine() n'est pas
6         // definie dans la classe Object
7 }
    
```

```

1 void afficheChaine(Object o) {
2     if (o instanceof MaClasse) {
3         MaClasse m = (MaClasse) o;
4         System.out.println(m.getChaine());
5         // OU System.out.println( ((MaClasse) o).↵
6         //     getChaine() );
7     }
8 }
    
```

Définition des attributs

Les attributs

Les données d'une classe sont contenues dans des variables nommées propriétés ou attributs.

Ces attributs peuvent être :

- 1 des variables d'instances,
- 2 des variables de classes
- 3 des constantes.

Définition des attributs

Les variables d'instances

- Déclaration de la variable dans le corps de la classe.
- Chaque instance de la classe a accès à sa propre occurrence de la variable.

```
1 public class MaClasse {  
2     public int variable1;  
3     int variable2;  
4     protected int variable3;  
5     private int variable4;  
6 }
```

Définition des attributs

Les variables de classe

- Les variables de classes sont définies avec le mot clé **static**
- Chaque instance de la classe partage la même variable.

```
1 public class MaClasse {  
2     static int  compteur;  
3 }
```

Définition des attributs

```

1  class ClasseTest {
2      int n;
3      double y;
4  }
5
6  public class MethodeStatic{
7
8      public static void main(String[] argv) {
9          ClasseTest objA1=new ClasseTest();
10         objA1.n+=4; // objA1.n vaut 4;
11         ClasseTest objA2=new ClasseTest();
12         // objA2.n = ? (0 ou 4)
13     }
14 }
    
```

Définition des attributs

```

1  class ClasseTest {
2      static int n;
3      double y;
4  }
5
6  public class MethodeStatic{
7
8      public static void main(String[] argv) {
9          ClasseTest objA1=new ClasseTest();
10         objA1.n+=4; // objA1.n vaut 4;
11         //equivalent a ClasseTest.n=4;
12         ClasseTest objA2=new ClasseTest();
13         // objA2.n = ? (0 ou 4)
14     }
15 }
    
```

Definition des attributs

Les constantes

- Les constantes sont définies avec le mot clé **final**
- leur valeur ne peut pas être modifiée une fois qu'elles sont initialisées.

```
1 public class MaClasse {  
2     final double pi=3.14 ;  
3 }
```

Les types primitifs

Généralités:

On distingue 4 catégories de types primitifs :

- Les entiers
- Les réels
- Les booléens
- Les caracteres

L'intervalle de valeurs représentables pour chacun des types peut varier en fonction de l'espace mémoire qu'ils occupent.

Les types primitifs: Les entiers

Les entiers

Les différents types d'entiers sont les suivants :

Nom	Taille	Intervalle représentable
byte	1 octet	$[-128 \dots 127]$ ou $[-2^7 \dots 2^7 - 1]$
short	2 octets	$[-32768 \dots 32767]$ ou $[-2^{15} \dots 2^{15} - 1]$
int	4 octets	$[-2^{31} \dots 2^{31} - 1]$
long	8 octets	$[-2^{63} \dots 2^{63} - 1]$

Les types primitifs: Les entiers

Les opérations sur les entiers:

On peut appliquer des opérateurs arithmétiques (+, -, *, /) à deux variables ou deux expressions de type entier.

Le résultat de cette opération est également du type entier.

NB:

Lorsque les deux opérandes sont de type entier, l'opérateur / calcule la division entière et l'opérateur % calcule le reste de cette division.

Les types primitifs: Les réels

Les réels

En Java il existe deux types de représentation pour les nombres réels : simple et double précision (respectivement les types float et double)

Les différents types de réels sont les suivants :

Nom	Taille	Intervalle représentable
float	4 octet	$[-3.4 \times 10^{38} \dots 3.4 \times 10^{38}]$
double	8 octets	$[-1.8 \times 10^{308} \dots 1.8 \times 10^{308}]$

NB: Lorsqu'au moins des opérandes est de type réel.

Les types primitifs: Les booléens

Les booléens

un seul type pour les booléens

Nom	Taille	Intervalle représentable
boolean	1 octet	[-true, false]

Les opérations sur les booléens:

On peut appliquer des opérateurs logiques (**et**, **ou**, **non**) à deux variables ou deux expressions de type booléens.

Le résultat de cette opération est également du type booléen.

Les types primitifs: Les booléens

Les opérateurs booléens:

- **&&** désigne l'opérateur ET logique
- **||** désigne l'opérateur OU logique
- **!** désigne l'opérateur NON logique

NB:

- La valeur renvoyée par tous les opérateurs de comparaison, comme (a < b) sont de type bool.
- Type requis par tous les opérateurs de contrôle de flux, comme if, while et do.

Ce que j'ai appris aujourd'hui

- ① La création et la manipulation des classes et objets en java.
 - déclarer une classe en java
 - créer un objet (instancier une classe)
 - utiliser la variable **this**
 - utiliser l'opérateur **instanceof**
 - utiliser l'objet **null**
 - déclarer les variables d'instances
 - déclarer les variable de classe
 - déclarer les constantes

Cours suivant

- 1 Nous continuerons avec Les classes et les Objet en Java